



資料分析與機器學習 (Data Analysis and Machine Learning)

Lecture 4 –
Logistic Regression & Matplotlib

教師：陳慶永

Outline

- The Problem of Adaline

(必須先吐槽一下 Adaline)

- Logistic Regression (邏輯回歸)

你以為這是吹風機，其實它是個刮鬍刀

- ▶ Conditional probability
- ▶ Activation function: sigmoid
- ▶ Loss function: cross-entropy

- Matplotlib

- Training a Logistic Regression Model

- Classwork

- ▶ Assignment #2





The Problem of Adaline

Issue of MSE for Classification

■ MSE

- ▶ Loss function adopted by Adaline

$$L(\mathbf{w}, b) = \frac{1}{2n} \sum_{i=1}^n \left(y^{(i)} - \sigma(z^{(i)}) \right)^2$$

$\{0,1\}$

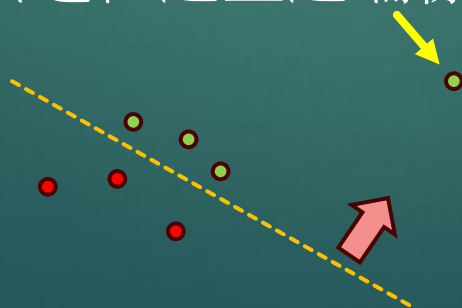
$$\sigma(z) = z = \mathbf{w}^T \mathbf{x} + b$$

$\mathbb{R} = (-\infty, \infty)$

- ▶ Very sensitive to outliers (異常値)

Issue of MSE for Classification

- 異常值懲罰過大導致錯誤的優化方向
 - ▶ 假設真實標籤 $y = 1$
 - ▶ 如果一個樣本的線性輸出 $\sigma(z) = z = 5$
 - 代表它被非常正確且有信心地分類為正類
 - ▶ 但 MSE 會計算其損失為 $(1 - 5)^2 = 16$ 😱
 - 這很荒謬！一個分類極度正確的樣本，竟然會產生巨大的損失
 - 這會逼迫決策邊界去迎合這些遠端樣本，進而破壞整體的分類線





Logistic Regression

Logistic Regression

■ Logistic Regression (邏輯回歸)

▶ A linear model for binary classification

- Despite its name, it is a model for *classification*, not regression

▶ One of the most widely used algorithms in industry

- Performs very well on linearly separable classes

▶ Can be generalized to multiclass settings

- Multinomial logistic regression (or softmax regression)
- Or using the technique of OvA (OvR)

We'll talk about different approaches to multiclass classification later (see pp. 20-21).

Conditional Probability (條件機率)

- Positive event

- ▶ Class label $y = 1$

- Conditional probability of positive event

- ▶ Prob{ An example belongs to class 1 given its features $\mathbf{x}$ }

$$p = p(y = 1 | \mathbf{x}) \quad [0,1]$$

- The odds (勝算、發生比) in favor of a particular event

$$\frac{p}{(1-p)}$$

(即「發生機率」除以「不發生機率」)
[0, ∞)

Logit (對數勝算)

■ logit function

- ▶ The **natural logarithm** of the odds (log-odds)

$$\text{logit}(p) = \log \frac{p}{(1-p)}$$

(即對「勝算」取自然對數 $\ln$)

- ▶ Domain (定義域): $[0,1]$
- ▶ Range (值域): $\mathbb{R} = (-\infty, \infty)$ 🤖



Convention in computer science: Without explicitly specifying the base, the notation $\log$ denotes the *natural* logarithm $\ln$.

Logistic Regression Model

- Assume that there is a *linear* relationship between the features and the log-odd for each sample

$$\text{logit}(p) = w_1x_1 + \dots + w_mx_m + b = \sum_{i=1}^m w_ix_i + b = \mathbf{w}^T \mathbf{x} + b$$

$\mathbb{R} = (-\infty, \infty)$

$$\text{logit}(p) = z \quad \text{(Net inputs)}$$

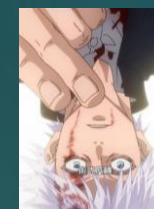
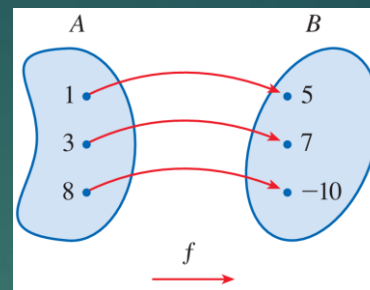
?

- We are interested in the **probability** p
 - What we need is the **inverse function** to map $z \in \mathbb{R}$ back to $p \in [0,1]$

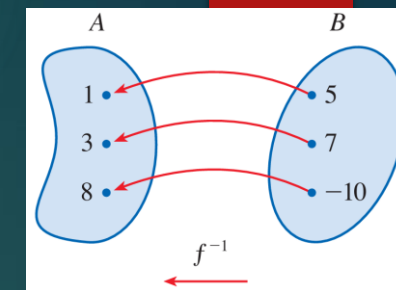
We'll talk about how to fit the parameters of the model later (see pp. 15-19).

Inverse Function

■ Definitions



(術式反轉 🕶️)



1 Definition A function f is called a one-to-one function if it never takes on the same value twice; that is,

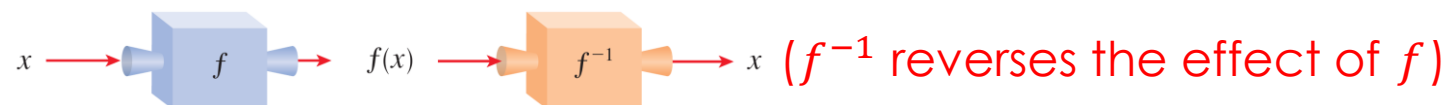
$$f(x_1) \neq f(x_2) \quad \text{whenever } x_1 \neq x_2$$

► Inverse function of f : f^{-1} (read “ f inverse”) (一對一函數才有反函數)

2 Definition Let f be a one-to-one function with domain A and range B . Then its inverse function f^{-1} has domain B and range A and is defined by

$$f^{-1}(y) = x \iff f(x) = y$$

for any y in B .



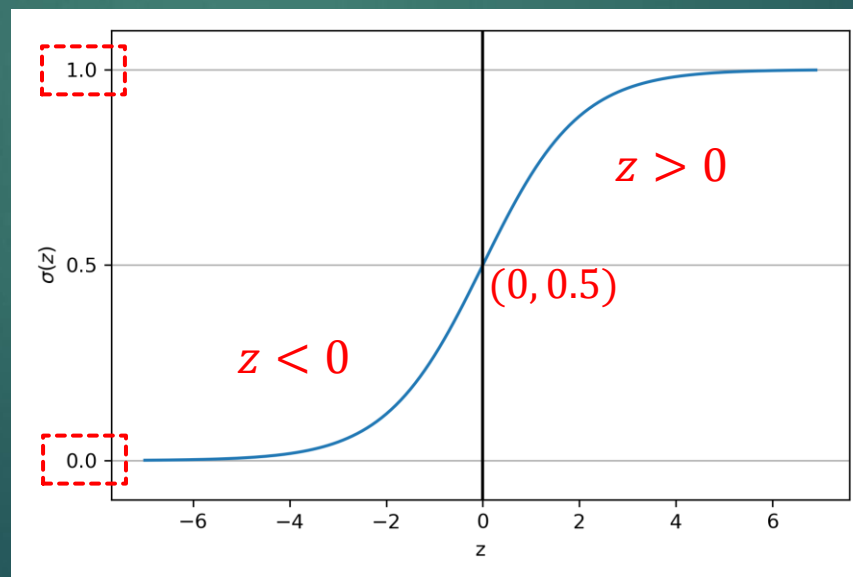
Sigmoid Function

■ Logistic sigmoid function (or sigmoid function)

► Inverse of the logit function

$$p = \sigma(z) = \frac{1}{1 + e^{-z}} \longleftrightarrow \text{logit}(p) = \ln \frac{p}{1-p} = z$$

► S-shaped curve



$$z = \mathbf{w}^T \mathbf{x} + b$$

Logistic Regression v.s. Adaline

■ Adaline

- ▶ Identity function as the activation function

$$\sigma(z) = z \quad (\text{線性輸出實數值})$$

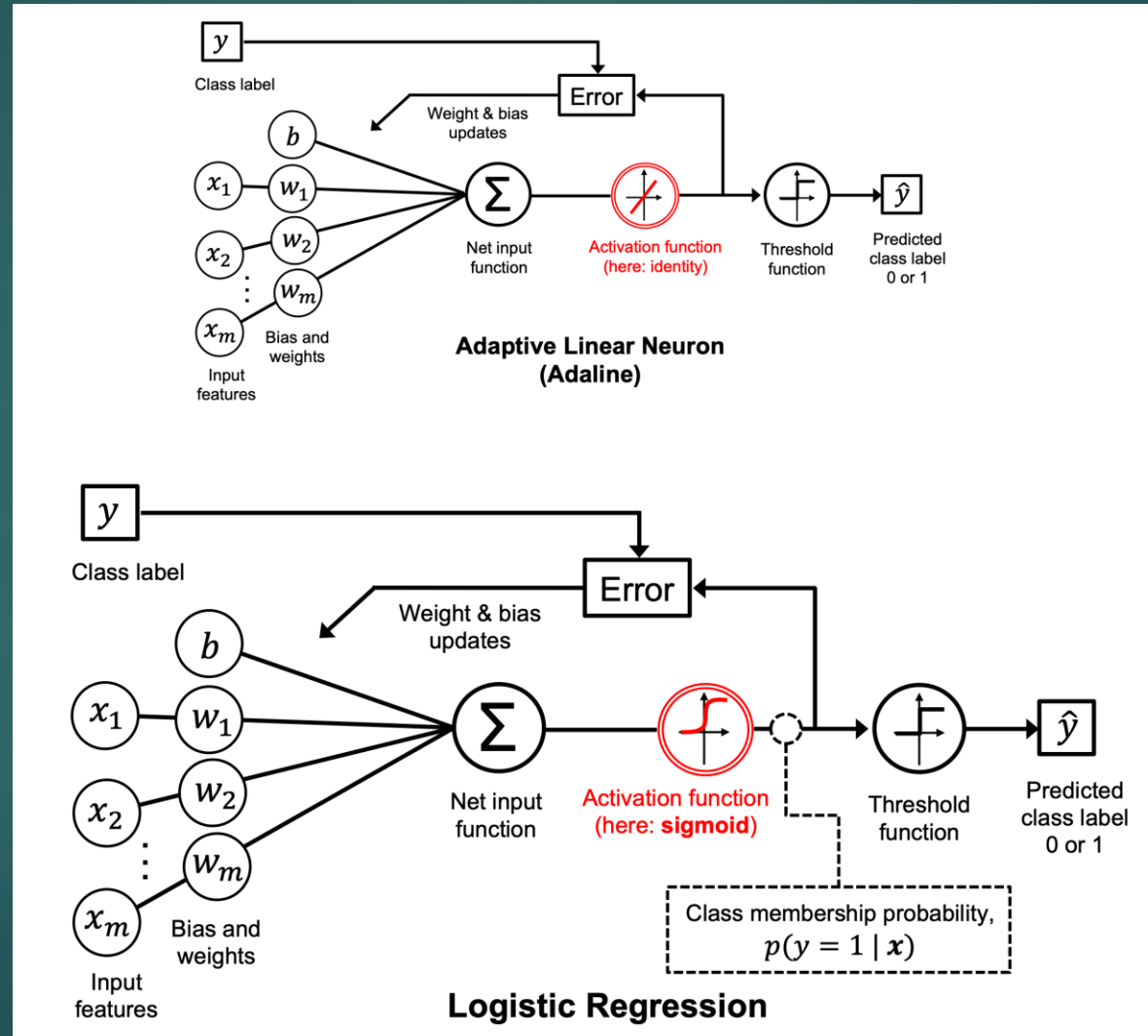
■ Logistic regression

- ▶ Sigmoid function as the activation function

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (\text{輸出機率值 } \text{😬})$$

In many applications, we are not only interested in the predicted class labels but also the estimates of the class-membership probability. (例如：雨天 & 下雨的機率)

Logistic Regression v.s. Adaline



$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(z) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Logistic Loss Function

■ Likelihood (概似、似然) function $\mathcal{L}$

$$\sigma(z^{(i)}) = p(y^{(i)} = 1 | \mathbf{x}^{(i)})$$

$$1 - \sigma(z^{(i)}) = p(y^{(i)} = 0 | \mathbf{x}^{(i)})$$

$$\mathcal{L}(\mathbf{w}, b | \mathbf{x}) = p(y | \mathbf{x}; \mathbf{w}, b) = \prod_{i=1}^n p(y^{(i)} | \mathbf{x}^{(i)}; \mathbf{w}, b) = \prod_{i=1}^n (\sigma(z^{(i)}))^{y^{(i)}} (1 - \sigma(z^{(i)}))^{1-y^{(i)}}$$

(assuming the individual examples are independent)

- ▶ 當資料 $\mathbf{x}, y$ 已固定時，不同參數 $\mathbf{w}, b$ 迎合 y 的可能性 (合理性)
 - 同時考慮迎合 n 筆資料的可能性
 - 在資料彼此獨立的假設下，該可能性 “A 且 B 且 C ... 發生的聯合機率” (joint probability) 即為 n 個條件機率的乘積
 - 可能性越高，表示該組參數越完美地迎合這些資料

Logistic Loss Function

■ Log-likelihood function (對 $\mathcal{L}$ 取自然對數)

$$l(\mathbf{w}, b|\mathbf{x}) = \log \mathcal{L}(\mathbf{w}, b|\mathbf{x}) = \sum_{i=1} [y^{(i)} \log(\sigma(z^{(i)})) + (1 - y^{(i)}) \log(1 - \sigma(z^{(i)}))]$$

- ▶ Reduces the potential of numerical *underflow* (e.g., $\mathcal{L} \approx 0$)
- ▶ Convert the product into a **summation** of factors
 - Easier to compute the partial derivatives of this function

1. $\log_b(xy) = \log_b x + \log_b y$

2. $\log_b\left(\frac{x}{y}\right) = \log_b x - \log_b y$

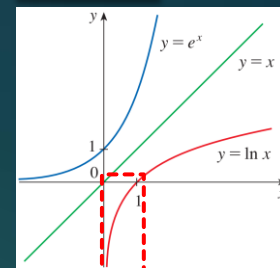
3. $\log_b(x^r) = r \log_b x$ (where r is any real number)

Logistic Loss Function

- We could use an optimization algorithm such as gradient ascent to **maximize** the log-likelihood
 - ▶ However, in ML, we used to **minimize** a loss function
- **Negative log-likelihood (NLL)** (對 log-likelihood 取負號)

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n [-y^{(i)} \log(\sigma(z^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(z^{(i)}))]$$

- ▶ Logistic loss function
 - Also called the binary **cross-entropy** (二元交叉熵 (尸尤))
 - To be minimized using **gradient descent**



$$\ln \underbrace{\sigma(z^{(i)})}_{(0,1)} \leq 0$$

Physical Meaning of the Loss

- Take a look at the loss for a *single* training example

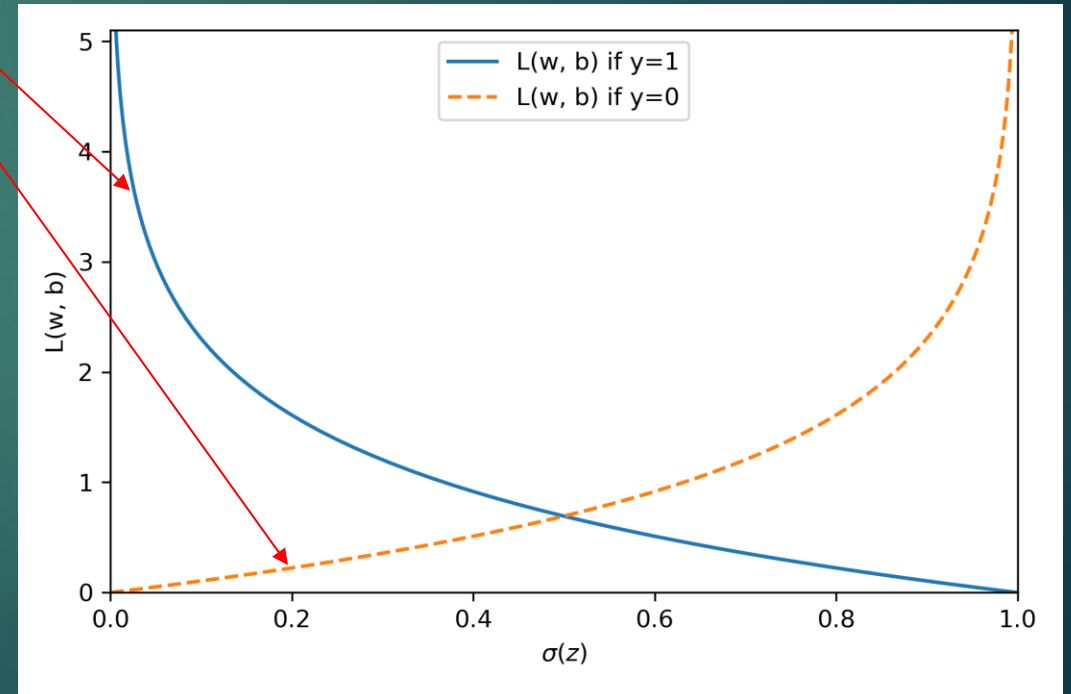
$$L(\sigma(z), y; \mathbf{w}, b) = -y \log(\sigma(z)) - (1 - y) \log(1 - \sigma(z))$$

- First (Second) term becomes zero *if* $y = 0$ ($y = 1$)

$$L(\sigma(z), y; \mathbf{w}, b) = \begin{cases} -\log(\sigma(z)) & \text{if } y = 1 \\ -\log(1 - \sigma(z)) & \text{if } y = 0 \end{cases}$$

Loss approaches 0 for correct prediction
Loss approaches ∞ for wrong prediction

It penalize wrong predictions with an increasingly larger loss



Partial Derivative of Log-likelihood Function

- Calculate the partial derivative of the log-likelihood function w.r.t the j th weight w_j

$$\frac{\partial L}{\partial w_j} = \underbrace{\frac{\partial L}{\partial a} \frac{da}{dz} \frac{\partial z}{\partial w_j}}_{\text{Apply chain rule}} \quad \text{where } a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

1) Derive terms separately: 2) Combine via chain rule and simplify:

$$\left. \begin{aligned} \frac{\partial L}{\partial a} &= \frac{a - y}{a - a^2} \\ \frac{da}{dz} &= \frac{e^{-z}}{(1 + e^{-z})^2} = a \cdot (1 - a) \\ \frac{\partial z}{\partial w_j} &= x_j \end{aligned} \right\} \rightarrow \frac{\partial L}{\partial z} = a - y \rightarrow \frac{\partial L}{\partial w_j} = (a - y)x_j = -(y - a)x_j$$

The same form as Adaline (but with different activation function)

$$w_j := w_j + \eta(y - a)x_j$$

Weight update: Take steps in the opposite direction of the gradient

We omitted averaging over the training examples for brevity.

Multiclass Classification

■ OvA (OvR)

- ▶ 模型會獨立訓練 K 個二分類器
 - 分類器 1 負責「類別 A vs 其它」
 - 分類器 2 負責「類別 B vs 其它」，以此類推
- ▶ 預測：輸入新資料時，讓 K 個分類器各自輸出機率，得分最高的那個類別就是預測結果

■ Multinomial (Softmax 回歸)

- ▶ 直接將二元邏輯回歸擴展到多分類
 - 不再使用 Sigmoid 函數，改用 Softmax 函數
 - 同時計算所有類別的得分，並將它們轉化為總和為 1 的機率分佈
- ▶ 預測：直接輸出全類別的機率，選機率最大者

Multiclass Classification

比較項目	OvR (One-vs-Rest)	Multinomial (Multiclass)
數學函數	Sigmoid (多個獨立計算)	Softmax (全類別同時計算)
模型權重	每個類別有一組獨立的權重	所有類別的權重在同一個損失函數中更新
機率解釋	各類別機率相加 不等於 1	各類別機率相加 嚴格等於 1
適合場景	類別數非常多、或想要快速平衡資料 多標籤 (multi-label) 分類	類別之間互斥 、追求更嚴謹的統計機率
計算速度	通常較快 (可平行運算多個二分類)	通常較慢 (每次反向傳播需考慮所有權重)

Matplotlib



Visualization

- Visualizing data is usually the best way to convey important engineering and science ideas and information
 - ▶ especially if the information is made up of many numbers
- Python has numerous graphics functions that enable you to efficiently display plots, surfaces, volumes, vector fields, histograms, animations, ...

Matplotlib

- **Matplotlib** is a comprehensive library for creating static, animated, and interactive visualizations in Python
 - ▶ [Matplotlib gallery](#)
 - ▶ [Matplotlib Quick Start Guide](#)

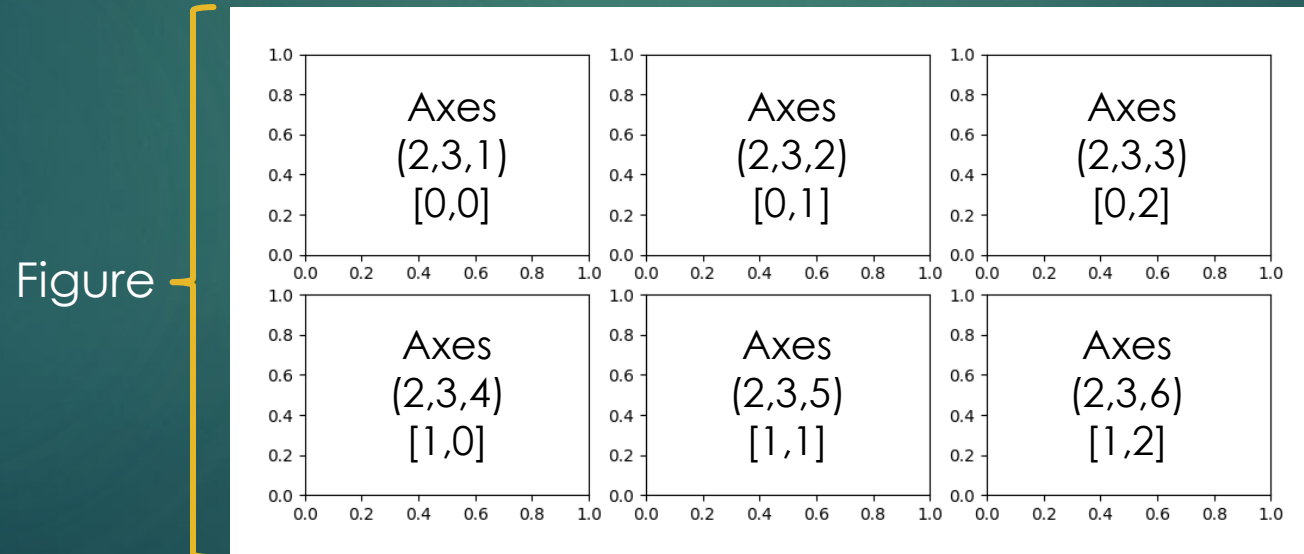
Standard Workflow

1. **Import Module:** Usually `pyplot` is imported as the alias `plt`.
2. **Prepare Data:** Use Python lists or NumPy arrays.
3. **Create Plot:** Call a plotting function (e.g., `plt.plot()`).
4. **Customize:** Add titles, labels, legends, etc.
5. **Show/Save:**
 - `plt.show()` to display the result, or
 - `plt.savefig()` to export it.

Key Concept

■ The Object Hierarchy

- ▶ **Figure:** The entire window or page where everything is drawn (the "canvas")
 - **Axes:** The area where the data is actually plotted (a single Figure can contain multiple Axes/subplots)



Parts of a Figure

■ Figure

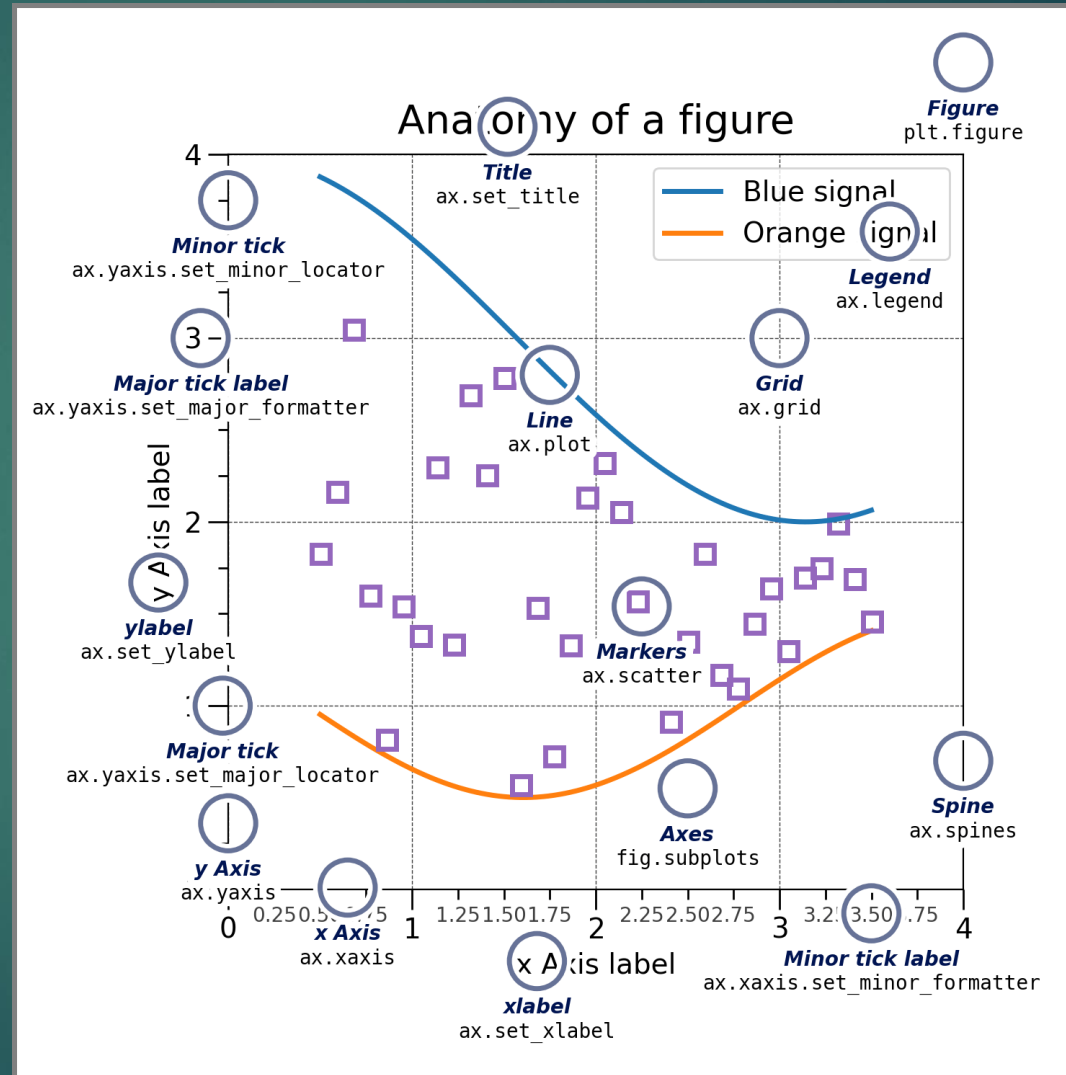
► Axes

● x-Axis

- Major tick
- Minor tick
- Major tick label
- Minor tick label

● y-Axis

- xlabel
- ylabel
- Title
- Legend
- Grid
- Spines



Two Interfaces

■ Implicit Interface

- ▶ pyplot-style (MATLAB-style)
- ▶ Function call of module: plt

■ Explicit Interface

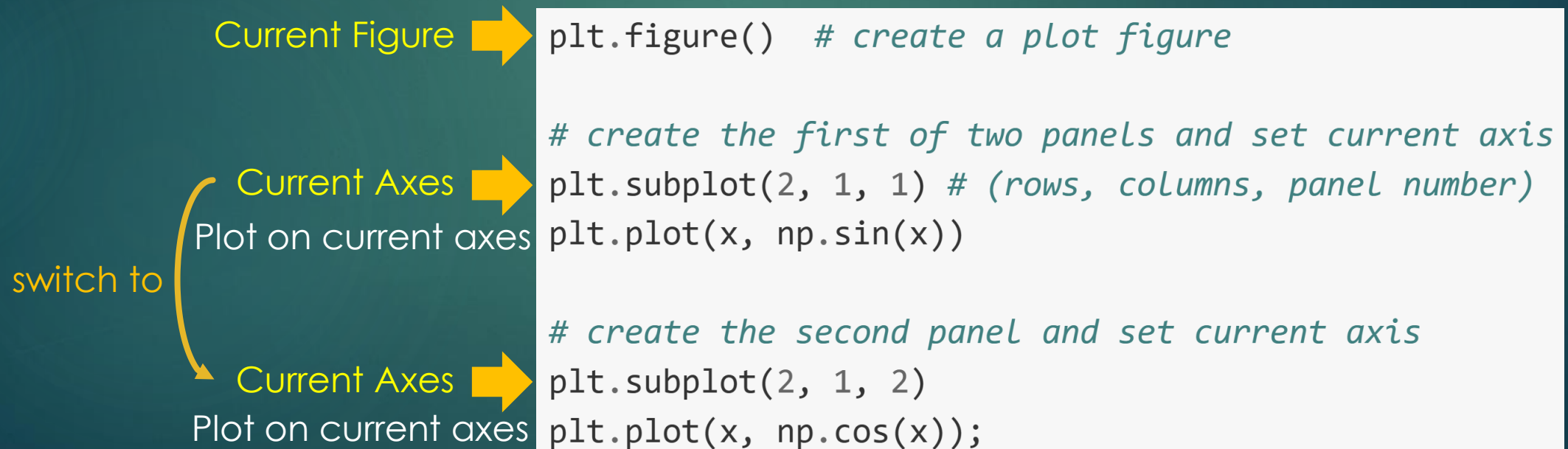
- ▶ Object-oriented-style
- ▶ Method call on object: axes

```
import matplotlib.pyplot as plt
# State-based Interface
plt.figure()
plt.subplot(2, 3, 3)
plt.plot(x, y)
plt.show()

# Object-oriented Interface
fig, axes = plt.subplots(2, 3)
axes[0, 2].plot(x, y)
plt.show()
```

Implicit Interface

- pyplot implicitly create & manage Figure and Axes
 - ▶ Keeps track of the "current (active)" figure and axes where all plt commands are applied
 - ▶ Fast and convenient for simple plots



Explicit Interface

■ Explicitly create Figure and Axes

- ▶ It doesn't matter whether Figure/Axes are active or not
- ▶ Just call methods on explicit Figure and Axes objects
- ▶ More flexible control over the plots

References to
Figure & Axes objects



Plot on specified Axes

```
# First create a grid of plots  
# ax will be an array of two Axes objects  
fig, ax = plt.subplots(2)  
  
# Call plot() method on the appropriate object  
ax[0].plot(x, np.sin(x))  
ax[1].plot(x, np.cos(x));
```

Quick Example (Line Plot)

■ Sample code: 04_1.py

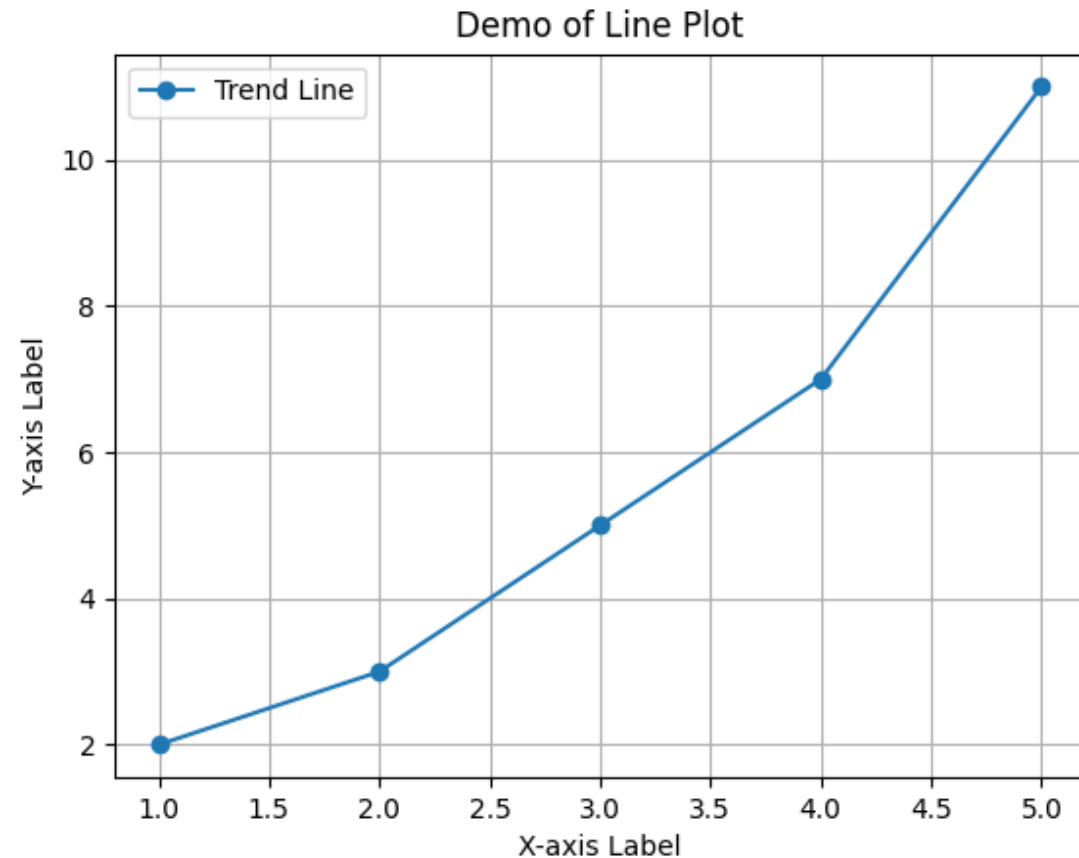
```
# 1. Import Module
import matplotlib.pyplot as plt

# 2. Prepare Data
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]

# 3. Create a Line Plot
plt.plot(x, y, marker='o', label='Trend Line')

# 4. Customization
plt.title('Demo of Line Plot')      # Title
plt.xlabel('X-axis Label')          # X-axis label
plt.ylabel('Y-axis Label')          # Y-axis label
plt.legend()                        # Show legend
plt.grid(True)                     # Enable grid

# 5. Display the plot
plt.show()
```



Plot Types

■ Overview

■ Common Plots

		Plot Type	Function	Purpose
Pairwise data	{	Line Plot	<code>plt.plot()</code>	Tracking data over time or sequences
		Scatter Plot	<code>plt.scatter()</code>	Observing correlations between two variables
		Bar Chart	<code>plt.bar()</code>	Comparing values across different categories
Statistical distributions	{	Histogram	<code>plt.hist()</code>	Visualizing data distribution
		Pie Chart	<code>plt.pie()</code>	Showing proportions of a whole

Common Functions

```
# Set style
plt.style.use('seaborn-v0_8-poster')
# Adjust the padding between and around subplots
plt.tight_layout()
# Display figure
plt.show()
# Save figure
plt.savefig("file_name.png")
```

Matplotlib Tutorials

■ [Tutorials from Matplotlib](#)

■ [W3Schools Matplotlib Tutorial](#)

Python Matplotlib

Matplotlib Intro

Matplotlib Get Started

Matplotlib Pyplot

Matplotlib Plotting

Matplotlib Markers

Matplotlib Line

Matplotlib Labels

Matplotlib Grid

Matplotlib Subplot

Matplotlib Scatter

Matplotlib Bars

Matplotlib Histograms

Matplotlib Pie Charts



Training a Logistic Regression Model

Mission

- Implement the logistic regression model in Python
 - ▶ The logistic regression and Adaline are very similar
 - ▶ Simple modifications to the Adaline implementation
 - Loss function: logistic regression loss (**cross-entropy**)
 - Activation function: **sigmoid** function
- Apply it to the Iris dataset (*two classes only*)
- Sample code: **04_2.py**
 - ▶ For full-batch GD as an example
 - ▶ The same changes could be made to the SGD version

Logistic Regression GD

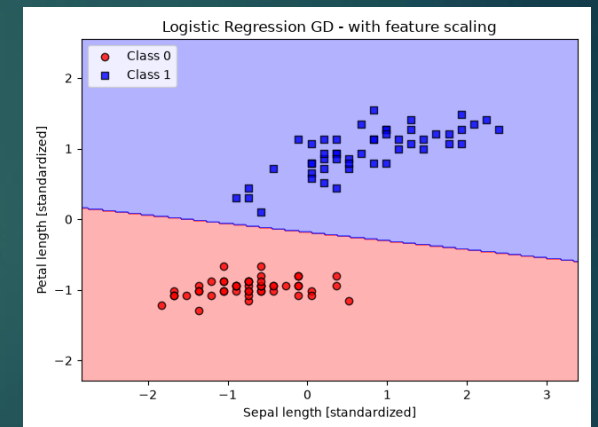
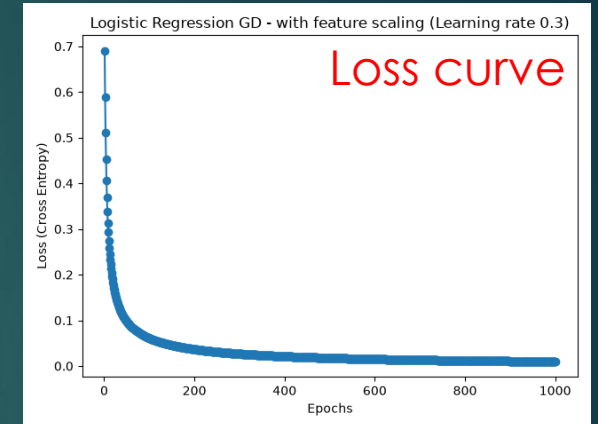
```
41 class LogisticRegressionGD:
42 >     """Gradient descent-based logistic regression classifier. ...
65 >     def __init__(self, eta=0.01, n_iter=50, random_state=1): ...
69
70     def fit(self, X, y):
71 >         """ Fit training data. ...
86             rgen = np.random.RandomState(self.random_state)
87             self.w_ = rgen.normal(loc=0.0, scale=0.01, size=X.shape[1])
88             self.b_ = np.float64 (0.)
89             self.losses_ = []
90
91             for i in range(self.n_iter):
92                 net_input = self.net_input(X)
93                 output = self.activation(net_input)
94                 errors = (y - output)
95                 self.w_ += self.eta * (X.T @ errors) / X.shape[0]
96                 self.b_ += self.eta * errors.mean()
97                 loss = (-y @ np.log(output) - ((1 - y) @ np.log(1 - output))) / X.shape[0]
98                 self.losses_.append(loss)
99             return self
100
101 >     def net_input(self, X): ...
104
105     def activation(self, z):
106         """Compute logistic sigmoid activation"""
107         return 1. / (1. + np.exp(-np.clip(z, -250, 250)))
108
109 >     def predict(self, X): ...
```

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n [-y^{(i)} \log(\sigma(z^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(z^{(i)}))]$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Logistic Regression GD

```
136 lrgd = LogisticRegressionGD(eta=0.3, n_iter=1000, random_state=1)
137 lrgd.fit(X_std, y)
138
139 plt.plot(range(1, len(lrgd.losses_) + 1), lrgd.losses_, marker='o')
140 plt.title('Logistic Regression GD - with feature scaling (Learning rate 0.3)')
141 plt.xlabel('Epochs')
142 plt.ylabel('Loss (Cross Entropy)')
143 plt.tight_layout()
144 plt.show()
145
146 plot_decision_regions(X_std, y, classifier=lrgd)
147 plt.title('Logistic Regression GD - with feature scaling')
148 plt.xlabel('Sepal length [standardized]')
149 plt.ylabel('Petal length [standardized]')
150 plt.legend(loc='upper left')
151 plt.tight_layout()
152 plt.show()
```





Classwork

Assignment #2

不要使用 scikit-learn 函式庫完成本作業

■ 目的：晶圓驗收分類器 (Wafer Acceptance Test Classifier)

■ 應用情境：

- ▶ 在半導體製造過程中，晶圓在完成關鍵蝕刻與沉積製程後，會進行晶圓驗收測試 (Wafer Acceptance Test, WAT)
- ▶ 工程師篩選出兩個對晶片良率影響至關重要的核心指標
 - 氧化層厚度偏差 (Oxide Thickness Deviation) (單位：Å (埃 = 0.1 nm))
 - 目標值為 0，偏差絕對值過大代表製程異常
 - 微量金屬污染度 (Metal Contamination) (單位：ppm)
 - 污染度越高，漏電流越大，晶圓越容易失效
- ▶ 目標是建立一個 **logistic regression model**，能自動將晶圓分類為「合格 (Pass)」或「不合格 (Fail)」

Assignment #2

不要使用 `scikit-learn` 函式庫完成本作業

■ 訓練集：`wat_train.csv`

▶ 15 筆歷史晶圓數據

- 特徵 x_1 ：氧化層厚度偏差 (Å)
- 特徵 x_2 ：微量金屬污染度 (ppm)
- 標籤 y ：晶圓狀態 (1: Pass, 0: Fail)

▶ 特徵工程思考：

- 觀察數據， x_1 (厚度偏差) 無論太正 (如 4.2) 或太負 (如 -4.0) 都會導致 $y = 0$
- 直接將 x_1 送入線性 logistic regression model 好嗎？😞
- 是否應先對 x_1 取絕對值 ($|x_1|$) 或平方 (x_1^2) 作為新的特徵？😲

	x_1	x_2	y
0	-1.2	0.5	1
1	0.8	1.2	1
2	-3.5	2.1	0
3	4.2	0.8	0
4	0.2	4.5	0
5	-0.5	1.1	1
6	1.5	1.8	1
7	-2.8	3.2	0
8	5.0	4.0	0
9	-0.1	0.9	1
10	2.0	0.4	1
11	-1.0	2.5	0
12	3.1	1.5	0
13	-0.7	1.4	1
14	-4.0	0.6	0

■ 使用各種訓練技巧：

▶ 特徵縮放 (Standardization)、嘗試不同的學習率

Assignment #2

不要使用 scikit-learn 函式庫完成本作業

■ 待驗收資料：wat_test.csv

▶ 5 筆全新的晶圓數據

	x1	x2
Wafer ID		
Wafer A	-1.1	0.9
Wafer B	4.5	0.3
Wafer C	0.0	4.8
Wafer D	-2.2	1.9
Wafer E	0.3	0.5

▶ 以訓練好的 logistic regression model 進行檢驗

- 檢驗資料需要進行「與訓練集一致的資料前處理」
 - 特徵轉換
 - 特徵縮放 (使用的是訓練集得出的統計值)

Assignment #2

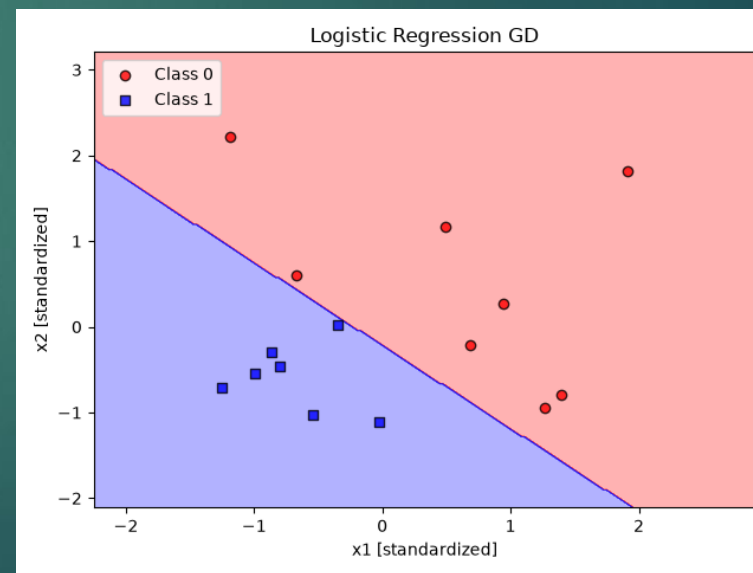
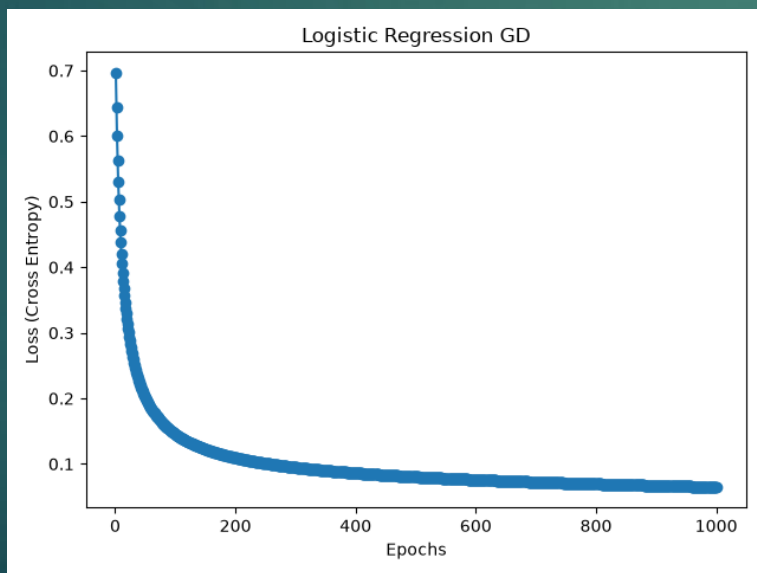
不要使用 scikit-learn 函式庫完成本作業

■ 預期結果 1

► 視覺化呈現模型訓練結果

- 損失曲線 (或訓練曲線) (Loss v.s. Epochs)
- 決策邊界區域與訓練集樣本的分佈

(從損失曲線只能判斷 loss 有收斂，但未經驗證無法判斷有無發生過擬合 (overfitting) 或訓練資料量是否足夠，後續會介紹如何正確地驗證與評估模型)



Assignment #2

不要使用 scikit-learn 函式庫完成本作業

■ 預期結果 2

▶ 以 DataFrame 格式輸出檢驗結果

- 待驗收的個別晶圓之通過機率
- 待驗收的個別晶圓之驗收結果

(這也是 logistic regression 的優點，除了分類結果，還提供置信度 (Confidence Level) 供參考)

Wafer ID		x1	x2	abs_x1	Pass Probability	Result
Wafer A		-1.1	0.9	1.1	99.65%	Pass
Wafer B		4.5	0.3	4.5	4.91%	Fail
Wafer C		0.0	4.8	0.0	0.06%	Fail
Wafer D		-2.2	1.9	2.2	9.55%	Fail
Wafer E		0.3	0.5	0.3	100.00%	Pass



Thank you